JAVA 2

Module ENU

Énumérations et records

Au programme

1. Des valeurs qui se comptent
2. Déclarer une énumération
3. L'ordre porte l'information
4. Comparer et aiguiller
5. Une enum est une classe
6. Random et `StringJoiner`
7. Les records

1. Des valeurs qui se comptent



Trois couleurs, pas une de plus

Un feu de signalisation a exactement trois couleurs

Avec des constantes `int`, rien n'empêche d'en demander une quatrième

Le compilateur ne voit rien, et l'erreur attend l'exécution

```
static final int VERT   = 0;
static final int ORANGE = 1;
static final int ROUGE  = 2;

feu.setCouleur(7);   // compile sans broncher
```

Il existe d'autres types que la classe

Type	Ce qu'il apporte	Où
`enum`	une liste de valeurs fermée	ce module
`record`	une donnée immuable, en une ligne	ce module
`abstract class`	un comportement partagé, incomplet	module `ABS`
`interface`	un contrat sans implémentation	module `ITF`
classe anonyme	une classe écrite sur place, sans nom	module `LAM`

2. Déclarer une énumération



Déclarer une énumération

Le mot-clé `enum` remplace `class`

Les constantes s'écrivent en majuscules, séparées par des virgules

Une variable de ce type n'accepte rien d'autre que ces valeurs

```
public enum CouleurDeFeu { VERT, ORANGE, ROUGE }

CouleurDeFeu couleur = CouleurDeFeu.VERT;
couleur = CouleurDeFeu.ROUGE;   // d'accord
couleur = 7;                    // refus a la compilation
```

Ce que toute énumération sait faire

Appel	Ce qu'il retourne	Exemple
`name()`	le nom de la constante	`"ROUGE"`
`ordinal()`	sa position, à partir de 0	`2`
`values()`	un tableau de toutes les constantes	`[VERT, ORANGE, ROUGE]`
`valueOf("VERT")`	la constante qui porte ce nom	`VERT`
`compareTo(autre)`	l'écart entre les deux positions	`-2`

L'ordre de déclaration des valeurs compte

Les méthodes `values`, `ordinal` et `compareTo` suivent l'ordre d'écriture

Cet ordre est celui du domaine, pas celui de l'alphabet

Insérer une constante au milieu décale toutes les suivantes

```
public enum CouleurDeFeu { VERT, ORANGE, ROUGE }

CouleurDeFeu.VERT.ordinal();       // 0
CouleurDeFeu.ROUGE.ordinal();      // 2
CouleurDeFeu.values().length;      // 3
```

3. L'ordre porte l'information



Démonstration



Démonstration

Le cycle sans conditions

Déroulé : Demo_ENU

Pour passer à la valeur suivante

La méthode `values` retourne le cycle des valeurs de l'enum

Le reste de la division ramène la dernière valeur à la première

Les parenthèses ne sont pas facultatives : l'opération `%` a priorité sur `+`

```
CouleurDeFeu[] cycle = CouleurDeFeu.values();

int suivante = (couleur.ordinal() + 1) % cycle.length;
couleur = cycle[suivante];
```

Ce que ce détour fait gagner

Une quatrième couleur s'ajoute sans qu'une ligne change ailleurs

Il n'y a plus de condition à lire, donc plus d'endroit où se tromper

En échange, l'ordre devient une information qu'il faut documenter

Exercices



ENU.1 Un feu de signalisation

fascicule ENU

4. Comparer et aiguiller : `switch`



Comparer deux valeurs

Chaque constante n'existe qu'en un seul exemplaire

L'opérateur `==` suffit donc, et il est sûr

La méthode `equals` fonctionne aussi, sans rien apporter de plus

```
if (couleur == CouleurDeFeu.ROUGE) {
    System.out.println("Stop");
}
```

Aiguiller : la forme classique

Les étiquettes s'écrivent sans le nom de l'énumération: le `switch` est fait sur jour de type `enum`, donc les valeurs sont connues.

Sans `break`, l'exécution continue sur le cas suivant

L'oubli du `break` est la faute la plus fréquente de cette forme

```
switch (jour) {
    case SAMEDI:
    case DIMANCHE:
        System.out.println("Repos");
        break;
    default:
        System.out.println("Travail");
}
```

Aiguiller : la forme à flèches

Si on utilise la flèche au lieu des deux-points, le `break` n'est plus nécessaire

On peut regrouper plusieurs étiquettes en utilisant une virgule

Un seul cas s'exécute, jamais celui qui suit

```
switch (jour) {
    case SAMEDI, DIMANCHE -> System.out.println("Repos");
    default               -> System.out.println("Travail");
}
```

Un `switch` qui retourne une valeur

Le `switch` peut aussi être utilisé comme une expression, il se termine alors par un point-virgule

Chaque branche fournit la valeur (elle ne l'affiche pas)

Si une branche nécessite plusieurs lignes, le mot-clé `yield` permet de retourner la valeur

```
String message = switch (jour) {
    case SAMEDI, DIMANCHE -> "Repos";
    case VENDREDI         -> "Presque";
    default               -> "Travail";
};
```

Tous les cas, sans oubli

Sur une énumération, le compilateur connaît la liste complète des cas

Une expression `switch` qui les couvre tous se passe de `default`

Ajouter une constante fait alors échouer la compilation, et c'est le but

```
String consigne = switch (couleur) {
    case VERT   -> "Passez";
    case ORANGE -> "Ralentissez";
    case ROUGE  -> "Arretez";
};
```

Exercices



ENU.2 Une température, deux unités

fascicule ENU

Nommer : `CC-MET.4`

5. Une enum est une classe



Une enum peut contenir des données

Chaque constante passe ses arguments au constructeur

Les champs et les méthodes s'écrivent comme dans toute classe

La constante répond alors elle-même aux questions qui la concernent

```
public enum Discount {

    BASIC(0.05), FIDELITY(0.075), SPECIAL(0.10);

    private final double taux;

    Discount(double taux) {
        this.taux = taux;
    }

    public double appliquer(double prix) {
        return prix * (1 - taux);
    }
}
```

Trois détails de syntaxe

La liste des constantes se termine par un point-virgule dès qu'il y a autre chose ensuite

Le constructeur est implicitement privé : un `new` sur une énumération est impossible

Les champs sont déclarés `final`, parce qu'une constante ne change pas

Un jeu de cartes, avec des enums

Avant de coder : qu'est-ce qui est fixé à l'avance dans un jeu de cartes ?



Les 4 couleurs et les 13 valeurs ne changeront jamais : deux énumérations, `Color` et `Value`

Chaque constante connaît son libellé : `PIQUE` porte « Pique », `AS` porte « As »

Une carte associe une couleur et une valeur : une classe `Card`, à deux champs

Le jeu, `Deck`, rassemble les 52 cartes, sans que 52 apparaisse dans le code

Le modèle du jeu

Pourquoi pas une seule énumération de 52 cartes ? C'est possible, mais il faudrait écrire les 52 constantes à la main

En séparant couleur et valeur, chaque carte sait de quoi elle est faite, et le jeu se construit par combinaison

```
Deck  ---[*]--->  Card  ---[1]--->  Color   PIQUE, COEUR, CARREAU, TREFLE
                        ---[1]--->  Value   AS, DEUX, ..., DAME, ROI
```

Toutes les combinaisons

Deux boucles `for-each` imbriquées parcourent toutes les paires

Le nombre de cartes n'apparaît nulle part dans le calcul

Ajouter une couleur ou une valeur suffit à changer la taille du jeu

```
Card[] cartes = new Card[Color.values().length * Value.values().length];
int i = 0;
for (Color couleur : Color.values()) {
    for (Value valeur : Value.values()) {
        cartes[i] = new Card(couleur, valeur);
        i++;
    }
}
```

6. Un détour par deux outils



Tirer un nombre au hasard

La classe `java.util.Random` produit des nombres au hasard

Sa méthode `nextInt(n)` retourne un entier de `0` inclus à `n` exclu

Un seul objet `Random` suffit pour tout un programme

```
Random hasard = new Random();

int de = hasard.nextInt(6) + 1;             // 1 a 6
int rang = hasard.nextInt(cartes.length);  // 0 a cartes.length - 1
```

Assembler un `String` avec un séparateur

La classe `StringJoiner` place le séparateur entre les éléments

Il n'y a plus de test sur l'indice, donc plus d'endroit où se tromper

Le constructeur accepte aussi un préfixe et un suffixe

```
StringJoiner texte = new StringJoiner(", ");
for (Card carte : cartes) {
    texte.add(carte.toString());
}
System.out.println(texte);
```

Exercices



ENU.3 Un jeu de cartes

fascicule ENU

Documenter : CC-COM.3

7. Les records



Une donnée, et rien d'autre

Le mot-clé `record` déclare une classe faite pour contenir des données

Les composants se déclarent entre parenthèses, comme des paramètres

Le corps du record reste souvent vide

```java
public record Point(int x, int y) { }

Point origine = new Point(0, 0);
System.out.println(origine.x());   // 0
```

Ce que le compilateur écrit

Le constructeur, avec les composants dans l'ordre déclaré

Un accesseur par composant, nommé `x()` et non `getX()`

Les méthodes `toString`, `equals` et `hashCode`

```
Point a = new Point(1, 2);
Point b = new Point(1, 2);

a.toString();   // "Point[x=1, y=2]"
a.equals(b);    // true
```

Un `record` ne change pas

Un record est `final` : aucune classe n'en hérite

Ses composants sont `final` : il n'existe aucun setter

Un record construit reste tel quel jusqu'à sa disparition

Ajouter une méthode

Un `record` accepte des méthodes, comme toute classe

Une méthode calcule à partir des composants du `record`

Ce qui se calcule n'est pas stocké

```
public record Rectangle(int largeur, int hauteur) {

    public int aire() {
        return largeur * hauteur;
    }
}
```

Valider à la construction

Le constructeur compact reçoit les composants avant qu'ils soient affectés. Remarquez l'absence de paramètres…

On peut protéger l'objet des valeurs absurdes par un test dans le constructeur (les exceptions seront vues au module EXC)

```
public record Rectangle(int largeur, int hauteur) {

    public Rectangle {
        if (largeur <= 0 || hauteur <= 0) {
            throw new IllegalArgumentException("dimensions positives");
        }
    }
}
```

Quand le record ne convient pas

Ce que vous modélisez	La forme
une adresse, un point, un résumé de partie	un `record`
un compte dont le solde change	une classe
un feu qui change de couleur	une classe
un objet dont d'autres doivent hériter	une classe

Exercices



ENU.4 Un record d'étudiant

fascicule ENU

Concevoir : CC-OBJ.2

Le fil rouge



G.6 Les énumérations `TypeJeu` et `ResultatPartie`
fil rouge GameHub

G.7 La partie connaît son type et son résultat
fil rouge GameHub

G.8 Le record `ResumePartie`
fil rouge GameHub

G.9 Choisir son jeu au lancement
fil rouge GameHub